DISEÑO SITIO WEB CSS

Un sitio web se divide en: ***Encabezado, menú de navegación, contenido principal y pie de página.***

- **Encabezado**: ubicado en la parte superior, suele contener un logotipo o el nombre de la web.

```
header {
  background-color: #f1f1f1;
  text-align: center;
  padding: 10px;
}
```

Resultado

My Header

- **Menú o Barra de navegación**: contiene una lista de enlaces para ayudar a los visitantes a navegar por su web.

```
/* Style the topnav */
ul.topnav {
  display: flex;
  list-style-type: none;
  margin: 0;
  padding: 0;
  background-color: #333333;
}

/* Style links in topnav */
ul.topnav li a {
  display: block;
  color: #f1f1f1;
  padding: 14px 16px;
  text-decoration: none;
}

/* Change color on hover */
ul.topnav li a:hover {
  background-color: #dddddd;
  color: black;
}
```

Resultado

Link1 Link2 Link3 Link4

- **Contenido**: la forma en la que se muestra el contenido de una web depende del dispositivo del usuario, los más comunes son:
    - Diseño de 1 columna (móviles)
    - Diseño de 2 columna (tablets y portátiles)
    - Diseño de 3 columna (pc de escritorio)

  Ejemplo diseño 3 columnas:

```
div.flex-container {
  display: flex;
  /* Show the flex items horizontally */
  flex-direction: row;
}

div.flex-container > div {
  margin: 10px;
}

/* Use media query and show the flex items vertically if screen width is less than 600px */
@media screen and (max-width:600px) {
  div.flex-container {
    flex-direction: column;
  }
}
```

Resultado

Column

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas sit amet pretium urna. Vivamus venenatis velit nec neque ultricies, eget elementum magna tristique.

Column

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas sit amet pretium urna. Vivamus venenatis velit nec neque ultricies, eget elementum magna tristique.

Column

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas sit amet pretium urna. Vivamus venenatis velit nec neque ultricies, eget elementum magna tristique.

- **Pie de página**: se encuentra al final de una página web. Suele contener información de autor y de contacto.

  Ejemplo pie de página básico:

```
footer {
  background-color: #f1f1f1;
  text-align: center;
  padding: 8px;
}
```

  Ejemplo pie de página fijo siempre visible en la parte inferior:

```
footer {
  position: fixed;
  bottom: 0;
  left: 0;
  width: 100%;
  background-color: #f1f1f1;
  padding: 8px;
  text-align: center;
  z-index: 1000;
}
```

NOVEDADES CSS

CSS está evolucionando para reemplazar muchos usos de JavaScript:

- Código más simple.
- Mejor rendimiento.
- Menos dependencias de JavaScript.
- Interfaces más accesibles.

1) Carruseles sin JS, Ahora se pueden crear carruseles con CSS gracias a nuevas propiedades como:

- `scroll-snap` → Permite deslizar imágenes.
- `overflow` → Permite la navegación entre elementos.
- `scroll-behavior` → Permite animaciones suaves.

Todo sin scripts.
Beneficios: menos código, menos errores, mejor rendimiento.

2) Se introduce el nuevo atributo `popover`, para crear elementos emergentes como:

- menús → abrir y cerrar popups.
- tooltips* → sin JS.
- ventanas flotantes → con comportamiento accesible incorporado.

*Un **tooltip** (o información sobre herramientas) es un mensaje descriptivo y breve que aparece cuando un usuario pasa el cursor sobre un elemento de la interfaz o pone el foco en él. Su función principal es ofrecer ayuda visual o contexto adicional sobre un botón, enlace o icono sin recargar la pantalla con texto innecesario (Por si os preguntabais como yo que coño era un tooltip).

3) Modales con `<dialog>`, permite crear ventanas modales directamente en HTML.
Características:

- Apertura con `showmodal()`*.
- Control de foco automático.
- Accesibilidad integrada.

*Un **showModal()** es un método nativo de JavaScript que se utiliza para abrir un elemento `<dialog>` como una ventana emergente de nivel superior (un modal).

4) Select personalizados con `selectmenu`, antes los `<select>` eran muy difíciles de estilizar y cada navegador los mostraba distinto. Las nuevas APIs permiten:

- Crear select con estilos personalizados.
- Mantener la accesibilidad.
- Evitar reconstruirlos con JS.

5) Mejor integración entre HTML y CSS permitiendo que el desarrollador:

- Escriba menos código
- Use más HTML y CSS
- Tenga componentes accesibles por defecto.

Contenido de Classroom del 17 de febrero y 13 de marzo (vídeo de YouTube + W3Schools).

TRANSICIONES EN CSS

Las **transiciones** permiten que un cambio de estilo ocurra de forma gradual en lugar de instantánea.

Propiedades importantes de `transition`:

- `transition-property`, define qué propiedad se anima.

```
transition-property: background;
```

- `transition-duration`, cuánto dura la transición.

```
transition-duration: 0.5s;
```

- `transition-timing-function`, define la velocidad del cambio. Valores comunes:
  - `linear`
  - `ease`
  - `ease-in`
  - `ease-out`
  - `ease-in-out`

  Esto afecta a cómo acelera o desacelera la animación.

- `transition-delay`, tiempo de espera antes de comenzar.

```
transition-delay: 0.2s;
```

El ***siguiente ejemplo*** muestra un elemento de 100 px * 100 px. Este elemento ha especificado un efecto de transición para la propiedad de ancho, con una duración de 2 segundos:

```
div {
  width: 100px;
  height: 100px;
  background-color: red;
  transition: width 2s;
}
```

Cómo desencadenar la transición

La transición se activa cuando se produce un cambio en las propiedades del elemento. Esto suele ocurrir dentro de pseudoclases (`:hover`, `:active`, `:focus` o `:checked`).

Cambiar varios valores de propiedades

Puede cambiar varias propiedades separándolas por comas.

El ***siguiente ejemplo*** agrega un efecto de transición para las propiedades de ancho, alto y color de fondo, con una duración de 2 segundos para el ancho, 4 segundos para el alto y 3 segundos para el color de fondo:

```
div {
  transition: width 2s, height 4s, background-color 3s;
}
```

ANIMACIONES EN CSS

Las **animaciones** en CSS permiten animar elementos HTML sin usar JavaScript, es decir, permite que un elemento cambie gradualmente de un estilo a otro, para ello se debe especificar algunos ***fotogramas clave*** (***keyframes***) para la animación. Los fotogramas clave definen los estilos que tendrá el elemento en determinados momentos.

Propiedades importantes de `animation`:

`animation-name`, especifica un nombre para la animación.

`animation-duration`, especifica la duración total de una animación. Si no se especifica, no se realizará ninguna animación, ya que el valor predeterminado es 0 s (0 segundos).

```
animation-duration: 2s;
```

`animation-iteration-count`, número de repeticiones.

```
animation-iteration-count: infinite;
```

`animation-direction`, dirección del movimiento. Valores:

- `normal`
- `reverse`
- `alternate`

`animation-fill-mode`, controla el estado final de la animación.

```
animation-fill-mode: forwards;
```

Regla CSS @keyframes

Cuando se especifican estilos CSS dentro de la regla, la animación cambiará gradualmente del estilo actual al nuevo estilo en determinados momentos. Para que una animación funcione, debes vincularla a un elemento.

El ***siguiente ejemplo*** vincula la animación "myAnimation" al elemento . La animación durará 4 segundos y cambiará gradualmente el color de fondo del elemento de "rojo" a "amarillo":

```
/* The animation code */
@keyframes myAnimation {
  from {background-color: red;}
  to {background-color: yellow;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: myAnimation;
  animation-duration: 4s;
}
```

En el ejemplo anterior hemos utilizado las palabras clave "desde" y "hasta" en la @keyframes regla, que representan el 0% (inicio) y el 100% (finalización).
También es posible usar porcentajes para agregar cambios de estilo.

El ***siguiente ejemplo*** cambiará el color de fondo del elemento cuando la animación esté completa en un 25%, en un 50% y nuevamente cuando la animación esté completa en un 100%:

```
@keyframes myAnimation {
  0%   {background-color: red;}
  25%  {background-color: yellow;}
  50%  {background-color: blue;}
  100% {background-color: green;}
}

div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: myAnimation;
  animation-duration: 4s;
}
```

El ***siguiente ejemplo*** cambiará tanto el color de fondo como la posición del elemento cuando la animación esté completa en un 25%, en un 50% y en un 100%:

```
@keyframes myAnimation {
  0%   {background-color:red; left:0px; top:0px;}
  25%  {background-color:yellow; left:200px; top:0px;}
  50%  {background-color:blue; left:200px; top:200px;}
  75%  {background-color:green; left:0px; top:200px;}
  100% {background-color:red; left:0px; top:0px;}
}

div {
  width: 100px;
  height: 100px;
  position: relative;
  background-color: red;
  animation-name: myAnimation;
  animation-duration: 4s;
}
```

Diferencia importante entre transición y animación:

CARACTERÍSTICA	TRANSITION	ANIMATION
Inicio	Ocurre cuando cambia una propiedad	Puede ejecutarse automáticamente
Complejidad	Simple	Más compleja
Estados	Inicio → Fin	Múltiples pasos

Las transiciones y animaciones se usan para:

- Efectos hover en botones.
- Menús desplegables.
- Elementos que aparecen en pantalla.
- Loaders o indicadores de carga.
- Micro-interacciones en interfaces.